

ブラウザネイティブの近距離ファイル転送

WebDrop

詳細ガイド

VERSION 1.0.87

WebDrop の仕組みを基礎から通しで解説します。3レーン構成、WebSocket/WebRTC、NAT/STUN/TURN/ICE、暗号化データチャネルと受信ラダー、超音波と動きによる近接セレモニー、予約型TDMAスケジューリング、近接スコアモデル、並行コホートの容量モデル、セキュリティモデル、そして画面カタログまでを収録しています。

日本語版

WebDrop とは

WebDrop はインストール不要の、ブラウザネイティブな近距離ファイル転送アプリです。AirDrop のような体験を目指します。ページを開くと、自分のアバターの周りに近くの端末が軌道状に表示されます。端末同士を軽くぶつける(またはQRを読み取る)とペアになり、以後はファイルが WebRTC 上で暗号化され、ブラウザからブラウザへ直接流れます。シグナリングサーバーは仲介に徹し、ファイルのバイトを一切見ません。

1.0.87

アプリ版

3

独立レーン

256 KiB

チャンクサイズ

500 MB

セッション上限

~50

同時ペア

製品を定義する3つの原則

信頼が先、操作は後

検証済みの接続が確立するまで転送 UI は出しません。送信・受信・チャットを初期画面に置くことはありません。

メタデータはシグナリング、バイトは WebRTC

WebSocket レーンは小さな JSON(在席・招待・近接セレモニー・SDP/ICE)だけを運びます。ファイルのバイトは暗号化された WebRTC データチャンネルを通り、サーバーを経由しません。

受信で驚かせない

受信したバイトは安全にバッファされ、ユーザーが「保存」を押したときだけダウンロードへ書き出されます。受信が勝手にダウンロードを始めることはありません。

本書の構成

- 3レーン構成と、転送が通しでどう進むか。
- 基礎からの解説: HTTP/WSS、WebRTC、SDP、NAT/STUN/TURN/ICE、DTLS/SCTP。
- データチャンネル、チャンク分割とバックプレッシャー、受信ストレージのラダー。
- 近接セレモニー: 超音波チャープ、バンプ/傾き、予約型TDMA、スコアリング。
- 並行コホートの容量モデル、セキュリティモデル、UIカタログ。

3レーン構成

WebDrop は3つの関心事を独立した「レーン」に分離します。これらを分けておくことが、低コスト・高プライバシー・高信頼の鍵です。

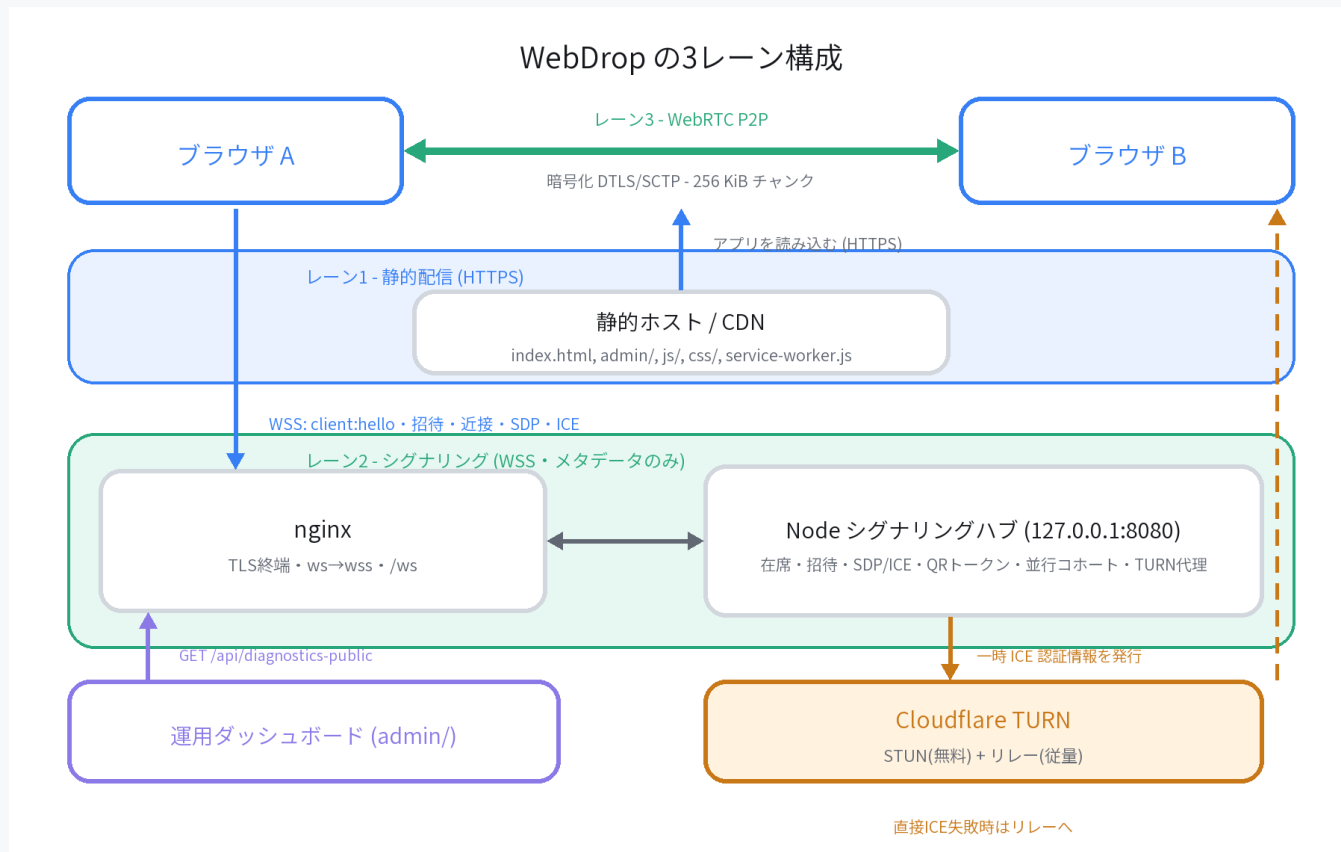


図1 - WebDrop の3レーン構成

- レーン1 - 静的配信(HTTPS): プレーンな HTML/CSS/JS を HTTPS で配信します。サーバーサイドレンダリングもバンドラもありません。マイク・モーション・Service Worker・WebRTC は secure origin でしか動かないため HTTPS は必須です。
- レーン2 - シグナリング(WSS・メタデータのみ): ws:// の WebSocket を nginx が wss://(TLS)へ昇格します。在席・招待・近接セレモニー・WebRTC ハンドシェイク(SDP+ICE)を選びます。ファイルのバイトは決して通しません。
- レーン3 - データ(WebRTC): 2つのブラウザ間の暗号化された RTCDataChannel。STUN で直接経路を見つけ、直接が不可能なときだけ Cloudflare TURN が中継します。

システム不変条件

シグナリングサーバーは仲介役であってファイルサーバーではありません。ファイルのバイトを受理・中継してはなりません。ペイロードに触れるのは2つのブラウザだけです。

転送の流れ(ステップ別)

同じインターフェースがローカルでも本番でも動きます。権限は「接続」操作からのみ要求し、転送ドックは検証済みの接続が成立した後にだけ現れます。

1. 発見。シグナリングが小さな在席情報を共有し、近くの端末が軌道に現れます。ファイル本体は送りません。
2. 選択と確認。送信者が候補をタップしてスワイプで接続します。権限(マイク/モーション)を要求するのはこの操作のときだけです。
3. ペアリングセレモニー。両端末が超音波チャープを送受信し、バンプと傾きを取得します。証拠が相互かつ整合し、タイミングも正しいときだけサーバーは身元を開示します。
4. WebRTC 交渉。シグナリングが SDP のオファー/アンサーと ICE 候補を中継し、WebRTC は直接経路を試し、必要なら TURN リレーへフォールバックします。
5. チャンク転送。ファイルを 256 KiB のチャンクに分割し、bufferedAmount を監視してバックプレッシャーを掛けながらデータチャンネルで流します。
6. 保存してから書き出し。受信チャンクは IndexedDB(iOS では上限付き Blob)に退避します。ユーザーが「保存」を押したときだけ StreamSaver でダウンロードへ流します。「転送完了」と「ダウンロード開始」は分離されています。

基盤: Web プラットフォーム

WebDrop は極小の静的アプリとして配布され、ブラウザ自身の機能に依存します。各基盤を平易に説明し、WebDrop が選んだ理由を添えます。

HTTPS 上の静的 HTML / CSS / JS

これは何か: ブラウザがダウンロードして実行する3つのテキストファイル - HTML(構造)・CSS(見た目)・JavaScript(振る舞い) - で、インストールは不要です。HTTPS はその通信を暗号化しただけのものです。飛び出す絵本のように、HTML はページ、CSS は挿絵、JS は仕掛けを動かすつまみです。

ブラウザがダウンロードして実行する3種のテキスト - 構造(HTML)、見た目(CSS)、振る舞い(JS) - で、インストールは不要です。index.html は固定的な DOM 契約で、アプリはルート要素の data-mode や data-theme などの属性を切り替え、CSS が反応します。理由: アプリを小さく高速に保て、強力なセンサー API のためにも HTTPS は必須だからです。

ES モジュール

これは何か: JavaScript 標準の仕組みで、コードをファイルに分け、互いに import / export します - それぞれが「提供するもの」と「必要なもの」を書いたラベル付きのレゴブロックのようで、ブラウザが組み合わせます。

`<script type="module">` によるネイティブな import/export です。js/app.js がストア・ビュー・サービス・トランスポート・コントローラを取り込みます(webpack/rollup なし)。import には ?v=1.0.87 というアプリ版に紐づくキャッシュバスターが付きます。理由: 小さな静的アプリにビルド工程は不要で、ネイティブモジュールは依存関係の境界を明確にします。

Service Worker とオフライン

これは何か: ページを閉じててもブラウザが動かし続ける小さな背景スクリプトで、ページのネットワーク要求に応答できます - よくある書類の控えを持つ受付係のようなもので、郵便局が閉まっても即座に渡してくれます。

ページを閉じててもブラウザが保持する背景スクリプトで、ネットワーク要求に応答できます。service-worker.js はアプリシェルの事前キャッシュし(CACHE_NAME = webdrop-v2-static-1.0.87)、runtime-config.js は常に最新を取得して稼働サーバー URL が古くならないようにします。理由: UI が即座に読み込まれオフラインでも動き、稼働サーバーを指す設定は最新に保たれます。

WebSocket と WSS/TLS 昇格

これは何か: WebSocket は1本の長寿命な接続で、両者がいつでも話せます - 何度も問い合わせる代わりに、はがきのやり取りではなく電話線を開いたままにする感じです。WSS はその暗号化(TLS)版で、ws:// に対する wss:// は、http:// に対する https:// と同じ関係です。

WebSocket は1本の TCP ソケット上の長寿命な双方向接続です。最初は「昇格」を求める通常の HTTP 要求として生まれ、サーバーが理解したことを示してプロトコルを切り替えます。wss:// では TLS が全体を包み、nginx が証明書を終端して、昇格済みソケットを 127.0.0.1:8080 の Node へ転送します。

```
GET /ws HTTP/1.1
Host: webdrop-wss-0618.japaneast.cloudapp.azure.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhLIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

101 の後、アプリが最初を送るフレームは client:hello でなければならず、サーバーは sessionId と一時的な turnAccessToken を付けて connected を返します。ハブは厳格です。バイナリフレームは拒否、ws は maxPayload = MAX_JSON_BYTES で生成、全メッセージをスキーマ検証し、IP 単位・クライアント単位のレート制限があります。理由: WebSocket は小さく頻繁な調整メッセージに最適ですが、ファイルレンではありません。

WebRTC・NAT・経路探索

WebRTC は2つのブラウザを直接つなぎます。WebDrop はそのデータ部分だけを使います。NAT 配下の2端末を接続させることが難問で、SDP・STUN・TURN・ICE はそれを解くためにあります。

WebRTC と SDP のオファー/アンサー

これは何か: WebRTC はブラウザ標準の仕組みで、2つの端末が間にサーバーを挟まず直接やり取りします - 音声・映像、ここでは生データです。SDP は最初に交換する短いテキストの「名刺」で、「自分が対応する内容と到達方法」を伝えます。

RTCPeerConnection が接続全体を管理します。能力の記述、候補の収集、経路の試験、データチャネルの保持です。SDP は小さなテキスト塊で「自分が対応する内容と到達方法」を表します。発信側は `createOffer()` -> `setLocalDescription` で送り、着信側は `setRemoteDescription` -> `createAnswer()` で返します。サーバーはこれらを `rtc:signal` として中継するだけです。SDP の `a=fingerprint` 行が後で暗号化チャネルを認証します。

NAT と、リレーが要る理由

これは何か: NAT(ネットワークアドレス変換)は、多数の家庭・オフィスの端末が1つのグローバルアドレスを共有し、その裏に隠れる仕組みです - 内線交換台を持つ1つの代表電話番号のようなものです。プライバシーには良い反面、外部から特定の端末へ直接「電話」できません。

NAT は1つのグローバル IP を多数の端末で共有し、要求していない受信を遮断するため、NAT 配下の2者は単純には接続できません。フルコーン/制限コーン NAT は同じグローバルポートを再利用し直接接続向きです。対称 NAT は宛先ごとに別のポートを割り当てるため穴あけが失敗し、これが TURN リレーへ落ちる典型的な理由です。

STUN・TURN・ICE

これは何か: STUN は「外から自分のアドレスはどう見えるか?」を手早く尋ねる方法です - 部屋の向こうの友人に名札を読んでもらう感じです。TURN は直接経路が無いときデータを中継するリレーで、荷物を転送してくれる郵便局のようなものです。ICE は全選択肢を試し、最良の有効な経路を選ぶ手続きです。

STUN は自分のグローバルアドレス(srflx 候補)を教えてください - 安価でステートレスです。TURN は直接経路が無いときにデータを中継するリレーで、認証情報が要り帯域を消費するため最終手段です。ICE はこれらを3段階でまとめます。host/srflx/relay 候補を収集し、接続性チェックを行い(送信プローブが応答に必要な NAT 穴を開けます)、最も優先度の高い有効なペアを採用します。WebDrop は GET /api/ice-servers から ICE サーバーを取得し、接続後 `classifyPathFromStats()` が経路を direct か relay と判定して、UI はリレー転送を 500 MB に制限します。

DTLS / SCTP - 暗号化データチャネル

これは何か: DTLS は TLS(HTTPS と同じ鍵マークの暗号)を、WebRTC が送る小さなパケット向けにしたもので、転送中の全バイトを暗号化します。SCTP はその上で、パケットを順序どおり欠落なく届けます。両者を合わせると、封緘され番号の付いた宅配袋のようなものです。

ICE が経路を選ぶと、ブラウザは DTLS ハンドシェイク(データグラム向けの TLS)を行い、証明書を交換して SDP に固定されたフィンガープリントを検証します - 中間者は割り込めません。続いて SCTP がその安全なチャネル上で動き、順序保証・信頼性・メッセージ枠付きの配送を提供します。こうして全バイトが自動で暗号化されます。それでも WebDrop はマニフェストとファイル別 SHA-256 を加えます。暗号化は経路を守りますが、ファイルの境界や完全性は守らないからです。

データチャネル・チャンク・バックプレッシャー

経路が確立すると、ファイルのバイトは RTCDataChannel を流れます - TCP のように順序保証・信頼性があります。WebDrop は制御メタデータとファイルのバイトを別々のチャネルに分け、互いをブロックしないようにします。

2本の順序付きチャネル + 256 KiB チャンク

これは何か: RTCDataChannel は WebRTC のパイプで、音声・映像だけでなく任意のデータを2つのブラウザ間で運びます - TCP のように順序保証・信頼性があります。「チャンク」は大きなファイルを一口大に切った1片で、本を一度にではなく1ページずつ送るようなものです。

WebDrop は2本の順序付きチャネル(webdrop-control-v1 と webdrop-file-v1)を開きます。送信側は各 File を 256 KiB のチャンク(DEFAULT_CHUNK_SIZE)に分割し、まずマニフェスト(ファイルID・サイズ・チャンク数・ファイル別 SHA-256)を送り、次に RTCDataChannel.bufferedAmount を監視しながらチャンクを流します。各セッションは 500 MB(DEFAULT_SESSION_CAP_BYTES)に制限されます。256 KiB は、モバイルで再送管理を軽く保つには十分小さく、メッセージ毎のオーバーヘッドを避けるには十分大きいサイズです。

バックプレッシャーの実際

これは何か: バックプレッシャーとは、受信側やネットワークのバッファが一杯になりかけたら送信を緩める仕組みで、あふれを防ぎます - コップに水を注ぐとき、縁からこぼれないよう一旦止めるのと同じです。

250 MB のファイルは 256 KiB チャンクで約1,000チャンクです。ループで send() を1,000回呼ぶと 250 MB 全部がブラウザの送信バッファに積まれ、タブが落ちかねません。代わりに送信側はハイウォーターマークで一時停止し、bufferedamountlow で再開します。常に少数のチャンクだけが送信中となり、ファイルサイズに関わらず RAM は一定に保たれます。

```
if (channel.bufferedAmount > HIGH_WATER) {
  await once(channel, "bufferedamountlow"); // let the network drain
}
channel.send(chunk);
```

受信ラダー

これは何か: IndexedDB はブラウザ内蔵の小さなデータベースで、端末上にデータを保存します。StreamSaver は受信バイトをブラウザ通常のダウンロードへ直接流し込みます。両者により、巨大ファイルを一度にメモリへ載せずに済みます。

受信側は数百MBのファイルを RAM 上で組み立てるのを避けねばならず、ウェブページは無断でディスクへ書き込めません。storage-client.js は次の優先順でラダーを上ります。

1. IndexedDB(対応する非iOSブラウザ): 受信しながら各チャンクを session/file/chunk をキーに書き込みます。ダウンロードはまだ始まりません。
2. 「保存」時、保存済みチャンクを自己ホストの StreamSaver ヘルパー経由でブラウザのダウンロードパイプラインへ流します。
3. iPhone/iPad Safari: 上限付き Blob フォールバックがメモリ上でチャンクを組み立て、Open(プレビュー)を提供します - メモリがペイロードに比例して増えるため上限付きです。
4. 互換フォールバック: IndexedDB が無いときは受信時に直接 StreamSaver へ書き込みます。安全に保持できるバックエンドが無ければ、バイトを受理する前にファイルを拒否します。

「転送完了」は「ダウンロード開始」ではない

受信チャンクはストレージに退避し、ユーザーが「保存」を押したときだけダウンロードへ書き出します。受信が勝手にダウンロードを始めることはありません。

近接センシング: 音と動き

ペアリングセレモニーは、2台の端末が本当に隣り合っている物理的証拠を集めます。マイクが聞き取れる近接超音波チャープと、加速度センサーが感じるバンプと傾きです。

超音波チャープ(Web Audio)

これは何か: Web Audio API は、ウェブページがスピーカーとマイクで音を生成・分析できる仕組みです。ここでの「チャープ」は近接超音波帯を掃引する短い音で、人にはほぼ聞こえませんが、近くの端末のマイクは拾えます - 2匹のコオロギが互いの鳴き声を認識するようなものです。

acoustic-proximity.js は近接超音波帯(DEFAULT_CHIRP: 18,600-19,400 Hz)で符号化チャープを発し、エコーキャンセル・ノイズ抑制・自動ゲインを切ってマイク録音します。ブラウザに高域エネルギーを消させないためです。音は近距離で指向性があるため、相手の符号化チャープが聞こえることは2台が物理的に近い良い証拠になります。

バンプと傾き(DeviceMotion)

これは何か: DeviceMotion は、端末の加速度センサー(どちらが下かを知り、揺れを感じる同じセンサー)をブラウザから読む API です。WebDrop はこれで、2台を物理的に軽くぶつけて傾ける動きを感じ取ります - 意図的で偽装しにくいジェスチャーです。

motion-proximity.js は、線形加速度が BUMP_ACCELERATION_THRESHOLD(10)以上、または重力ベクトル変化が 3.5 以上のときバンプを、 $|\beta|$ か $|\gamma|$ が 30 度を越えたとき傾きを検出します。これらは近接スコアに寄与し、サーバー側の必須証拠でもあります。端末を軽くぶつける動作は意図的で、偶然には起こりにくいものです。

検出は音量ではない

これは何か: 相関とは、「録音が、送った正確なパターンにどれだけ一致するか」を測る計算で(1.0=完全一致)、「どれだけ大きかったか」ではありません - 騒がしい部屋で、ただの雑音ではなく特定のメロディを聞き分けるのに似ています。

検出は「大きかったか」ではなく「録音にこの特定の符号化スイープが、背景より明らかに強く含まれていたか」です。相関は期待テンプレートを録音に重ねて一致度を測り(1.0=完全)、エネルギーマージンはチャープ帯域のエネルギーを近傍背景と dB で比較します。WebDrop は段階的に受理します(例: 相関 0.30 / 0.20 / 0.16 を 4.5 dB / 8 dB のマージンで裏付け)。サーバーは共有帯域を ~ サンプルレート x 0.45 - 100 Hz 未満かつ 420 Hz 以上の帯域幅に保ち、ナイキスト限界を超える音を端末に検出させません。

QR は確実なフォールバック

マイク/モーションが拒否・非対応のときは、一方が一度きりのサーバー発行 QR トークンを表示し、他方が読み取ります - 必要なのはカメラと画面だけです。

予約型TDMA: 1つの窓に事前割当スロット

近くの複数端末が同時にチャープすると音が衝突し、誰もきれいに復号できません - 古典的な多元接続問題です。サーバーは既にコホートを把握しているので、誰かが送信する前に時間スロットを割り当てます。



図2 - 予約型TDMA スケジュール

これは何か: TDMA(時分割多元接続)とは、複数の送信者が1つのチャネルを、それぞれ自分の時間スロットで順番に使って共有する方式です - 会議で全員が同時にではなく一人ずつ話すと決めるのに似ています。

これは予約型TDMA(時分割多元接続)です。各端末は 3,600 ms の窓全体を録音しますが、送信は自分のスロットだけで行い、後で全員を復号します。Aloha 系です - スロット化 Aloha は端末がスロットに賭けて衝突時に後退しますが、WebDrop はスロットを中央で予約します。これは小規模で既知、かつサーバー協調のコホートでは確実に信頼性が高い方式です。

- スロット下限: 符号化チャープには ~520 ms + ~80 ms のガードが要り、1スロット ~600 ms。
- $\text{floor}(3,600 / 600) = 6$ スロットが入るため、コホート上限がちょうど6です。
- 大きなコホートには長い窓(PROXIMITY_SESSION_DURATION_MS)が必要で、数字を増やすだけでは不可です - 上限はスロット下限の天井にクランプされます。

スコアリング: 相互署名と勝者マージン

ペアの同定は「誰が誰の符号化チャープを聞いたか」で決まります - 音量や「誰が近い」ではありません。これが A<->B と C<->D の誤ペアを防ぎます。

相互符号化署名(主判定)

A<->B では、A は自分の割当署名を報告し、かつ B の署名を聞いたと報告し、B も A について同じことを行わねばなりません - いずれも検出しきい値以上(hasReciprocalAcousticEvidence)です。A は B の署名を聞いており C のではないので、A+C は決して一致しません。秘密の握手は、両者が自分の半分を行い、かつ相手の半분을認識したときだけ成立します。

勝者マージン(フェイルセーフ)

勝者マージンは、最良一致の署名が次点より十分に明確であることを要求し、近くの第三の端末を相手と取り違えないようにします。ガードはフェイルセーフです。acousticConfidenceMargin が欠落・非有限なら、既定で通過させず検査を失敗させます(ACOUSTIC_WINNER_MARGIN = 0.04)。

検証に必要な条件

条件	ルール
スコア	近接スコア 55% 以上
証拠	超音波 + バンプ + 傾き すべて必須
相互性	互いの署名を聞いていること
タイミング	バンプ時刻が 4,000 ms 以内・有効なセレモニー
勝者	マージンを超える明確な勝者

傾きは存在確認のみです。存在は必須ですが、どの端末同士がペアになるかは決めません。正確なバンプ時刻はゲートかつ同点処理であり、同定そのものには使いません。

並行コホートの容量モデル

1つのグローバルなペアリングセッションの代わりに、ハブは多数の並行する有界コホートを走らせます。6人の小部屋を同時に多数、グローバルな人数上限の下で運用します。

新規参加者は空きのあるコホートに入るか、新しいコホートを開きます。コホートが満員(6)になると閉じてセレモニーを開始します。MAX_TOTAL_PROXIMITY_PARTICIPANTS(既定100)が全体を縛り、それを超える参加は、どのコホートも変更する前に reason: capacity_reached できれいに失敗します。よって $100 / 6 = \text{約}17$ コホート = 最大 約50ペアです。コホート上限 MAX_PROXIMITY_SESSION_CLIENTS(既定6)はスロット下限の天井にクランプされ、スケジューラは下限未満の音響スロットを作れません。

調整ノブ(と各々のトレードオフ)

ノブ	既定	上げると...
MAX_TOTAL_PROXIMITY_PARTICIPANTS	100	並行参加者が増える。意味を持つには Redis + 複数ノードが必要
MAX_PROXIMITY_SESSION_CLIENTS	6	窓を広げない限り無効(スロット下限にクランプ)
PROXIMITY_SESSION_DURATION_MS	3,600 ms	スロットが増え大きなコホートが可能に。セレモニーは遅くなる
ACOUSTIC_SESSION_STAGGER_MS	600 ms	同時開始のコホートをずらし、一斉チャープを避ける
ACOUSTIC_MAX_CONCURRENT_SUBBANDS	4	コホートを周波数レーンに分割。帯域を広げるまでは無効

正直な注意点

ソフトウェアは ~50 の近接ペアを許容しますが、1部屋で ~800 Hz 帯域を共有する ~50 ペアは音響的に混雑します - 実際の信頼性は端末依存の問題です。10,000 への道は、設定 + Redis/共有プレゼンス + スティッキーな複数ノード WS + 段階的負荷試験です。

複数端末ペアリング Q&A

多数の端末を同時にペアリングする際に最もよく聞かれる質問と、現行コードの実際の挙動です。

● A+B と C+D - A+C はどう防ぐ？

主判定は相互署名であり、誰が近いかではありません。A は B の符号化チャープを、B は A のものを聞いたので A+C は一致しません。バンプ時刻(4,000 ms 以内)はゲートと同点処理のみです。

● 片方が遅れて接続を押したら？

コホートの参加窓は 1,800 ms で、1回 1,800 ms 延長されます(合計 ~3.6 秒)。その間なら同じコホートに、開始後なら別の並行コホートに入ります。1台だけ残ると no_nearby_partner で失敗します。

● 別室のグループは干渉する？

しません。論理面: ペアリングは相互符号化署名でゲートされ、迷い込んだチャープは誤った署名なので一致に失敗します。物理面: 超音波は指向性が高く減衰が速く、コホートは開始をずらしサブバンド化されます。

● 上限は? 上限到達時の表示は?

現在の硬い天井はセレモニー同時参加 100 人(~50 ペア)で、単一ノードのメモリ上限です。上限時に接続を押した端末は capacity_reached を受け取り、再試行か QR を使えます - 誤ったペアリングは起きません。

UI状態機械と操作ゲート

インターフェースは小さな明示的状态機械です。最も重要な原則は、転送ドックが検証済みの接続が成立した後にだけ存在することです。

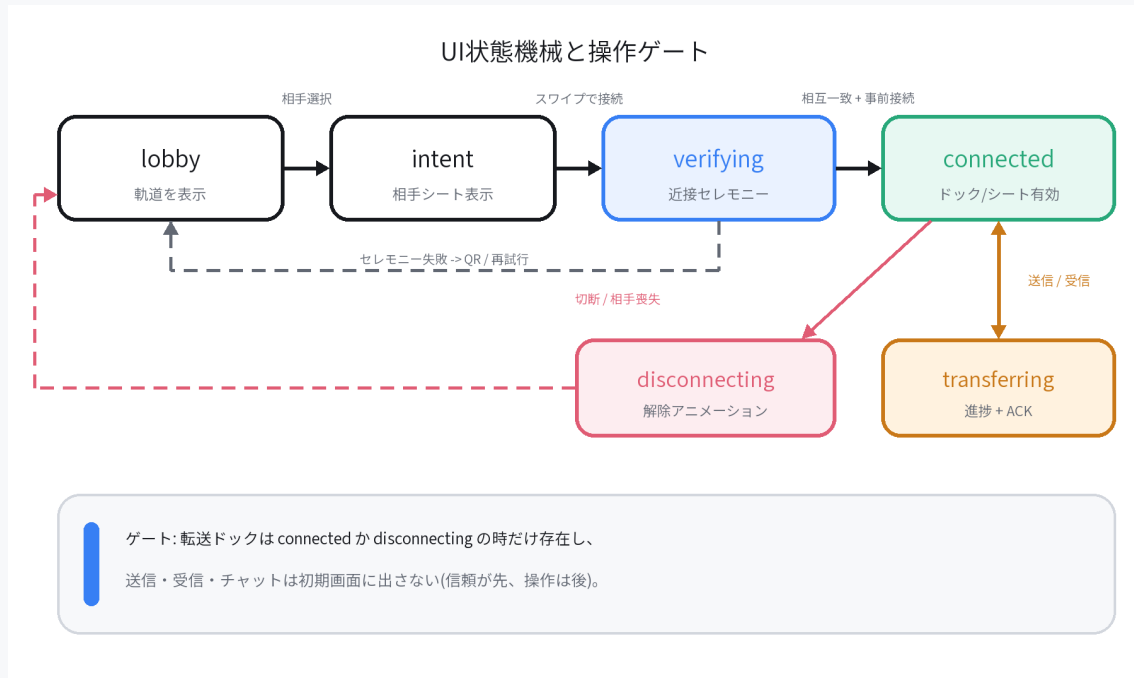


図3 - UI状態機械と操作ゲート

ランタイムはルート要素の data-mode を lobby -> intent -> verifying -> connected -> disconnecting と遷移させます。転送は独自のモードではなく進捗オブジェクトとして追跡します。AppView.renderTray() のゲートは「信頼が先、操作は後」を強制する実装です。

```
const connected = state.mode === "connected" || state.mode === "disconnecting";
this.nodes.tray.hidden = !connected; // the dock exists only after a verified link
```

信頼が先、操作は後

送信・受信・チャットを初期画面に置くことはありません - ドックは相互かつ近接検証済みの接続が成立して初めて現れます。

セキュリティモデル

クライアントが信頼するのは、現在のタブでの直接のユーザー操作と、ユーザーが選んだファイルハンドルだけです。相手が申告した名前や MIME タイプ、想定されたペアリングセッションやスキーマ外のメッセージ/SDP/ICE は信頼しません。

フロントエンドの堅牢化

- XSS安全プレビュー: 相手申告の危険な MIME(text/html・image/svg+xml・各種 text/*・XML)はダウンロード専用で、ページ内では開きません。プレビュー可能なのは画像・動画・PDF のみです。
- オブジェクトURLのリークなし: 受信ファイルのオブジェクトURLは使用後に失効させ、ファイル名は表示前にエスケープします。

バックエンドの堅牢化

- 許可オリジンを強制し、バイナリフレームを拒否、ペイロードを上限化(ws maxPayload = MAX_JSON_BYTES)し、全メッセージをスキーマ検証します。
- IP単位・クライアント単位でレート制限し、一時的な TURN 認証情報のみを発行、ログ内の機密(authorization・token・credential・iceServers・sdp)を秘匿します。
- 近接解析が有効なとき、両者が verified に至るまで RTC/チャット/経路/転送のルーティングを遮断します(スコア 55% 以上 + 必須証拠)。

診断アクセス

運用ダッシュボードは1つの認証付きエンドポイント GET /api/diagnostics-public だけを読みます。METRICS_API_TOKEN のベアラーが必要です。ペイロードはメタデータのみで、TURN 認証情報・QR トークン・生のマイク音声・ファイルのバイトは一切含みません。

運用ダッシュボード

admin/index.html(diagnostics-api.js 経由で readiness.js が駆動)は、運用者に対しシステム健全性とライブのペアリング計測を読み取り専用で提供します。自分のマイクを開くことはありません。

● Readiness タブ

/readyz からの環境・健全性サマリー - 稼働環境が実際に持つ機能と認証情報を示します。

● ライブテスト タブ

/api/diagnostics-public からの有界な端末/ペア/コホート計測と、admin:monitor:* による単一端末の音響診断です。

admin/diagnostics.html は現在 admin/index.html?tab=live へのリダイレクトです。ダッシュボードはメタデータのみを表示するため、運用者はファイルのバイトや認証情報に一切触れずに準備状況とライブコホートを監視できます。

制限と正直な注意点

WebDrop は意図的に範囲を絞っています。以下は現在の実際の制限で、期待がコードと一致するよう率直に記します。

- セッション上限: 各送受信セッションは 500 MB(DEFAULT_SESSION_CAP_BYTES)に制限され、リレー(TURN)経路も同じ天井に従います。
- 並行性: 単一メモリノードでセレモニー同時参加 ~100 人(~50 ペア)。10,000 到達には Redis + 複数ノードのスティッキー WS + 負荷試験が必要です。
- 音響の信頼性は端末依存です: 帯域・ゲイン・スロット長・向きの影響は、音響を変更するたびに実機で再測定する必要があります。
- プラットフォーム制限: マイク/モーション/WebRTC には HTTPS(secure origin)が必須です。iOS Safari は大きな受信に IndexedDB ではなく上限付きのメモリ Blob を使います。

上限でも壊れない

天井が満員のときに接続を押した端末は、どのコホートも変更される前に capacity_reached を受け取ります。少し待って再試行するか QR にフォールバックできます。100 は安全な既定値であって製品上の限界ではありません。

本番化に向けて

静的クライアントとシグナリングバックエンドは実装・堅牢化済みです。残るのは運用面の作業です。実機で近接セレモニーを較正し、直接接続とリレー転送を規模ありで実証し、10,000ユーザー目標に向けて容量を拡張します。

● 実機での較正

実機の iOS/Android で超音波検出・バンプ・傾きを測定し、帯域・ゲイン・スロット長・しきい値を調整します。音響の信頼性は端末依存です。

● 転送の実証

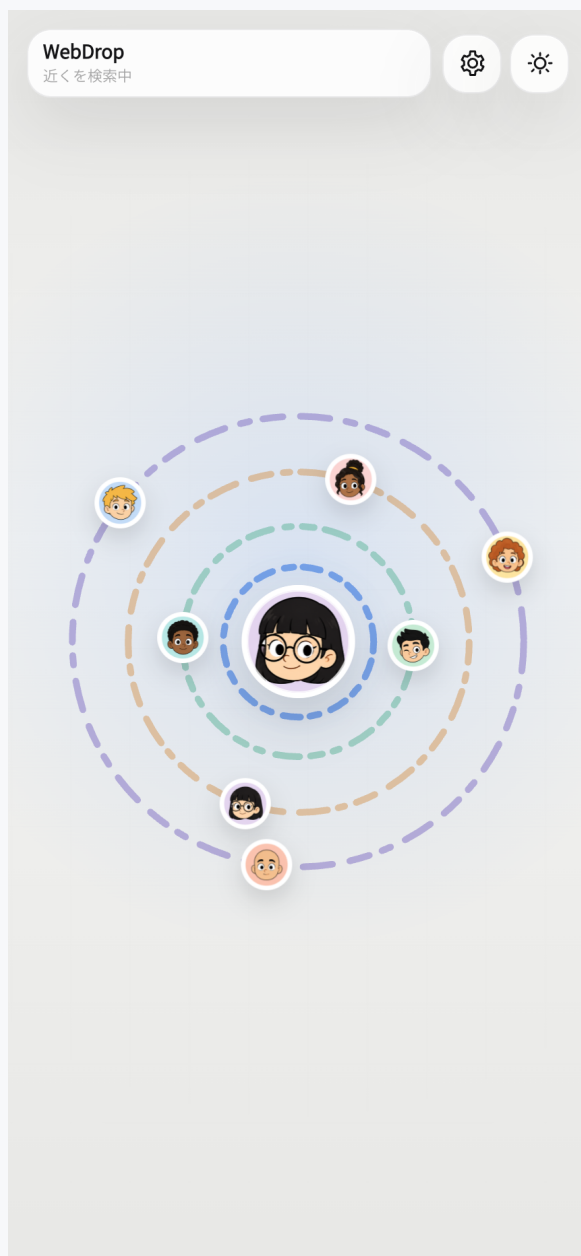
2つのブラウザ間で直接および TURN リレーの WebRTC 転送を実証します(大容量受信・キャンセル/再試行・保存容量不足を含む)。

● 10,000へのスケール

共有在席/状態(Redis)、スティッキーなマルチノード WebSocket 分散、段階的負荷試験、監視を整えてから全体参加上限を引き上げます。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧

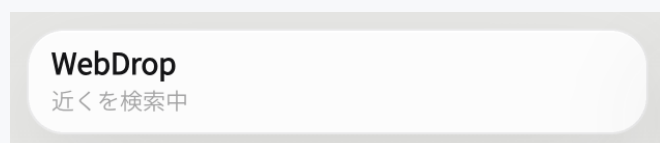


ライトモードのホーム画面

タイトル、端末名、4本の軌道、緩やかに表情が変わる自分のアイコン、静止した近隣候補を表示するメイン画面です。

画面カタログ

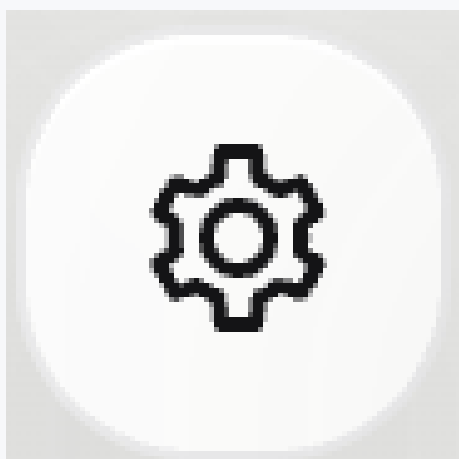
承認済み WebDrop 1.0.87 画面一覧



02

WebDropステータス

転送操作を早く見せすぎず、WebDrop名と近隣検索中または接続中の状態を表示します。



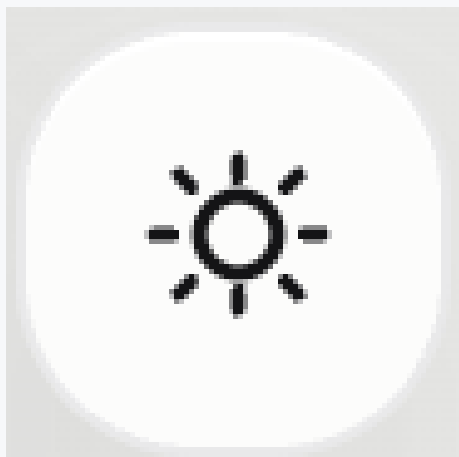
03

設定アイコン

プロフィール、リング色、言語、アプリ情報、バージョンを設定するシートを開きます。

画面カタログ

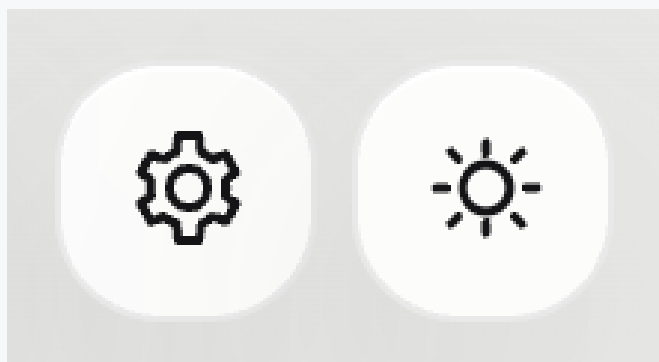
承認済み WebDrop 1.0.87 画面一覧



04

テーマ切替

メイン画面からライトモードとダークモードを切り替えます。



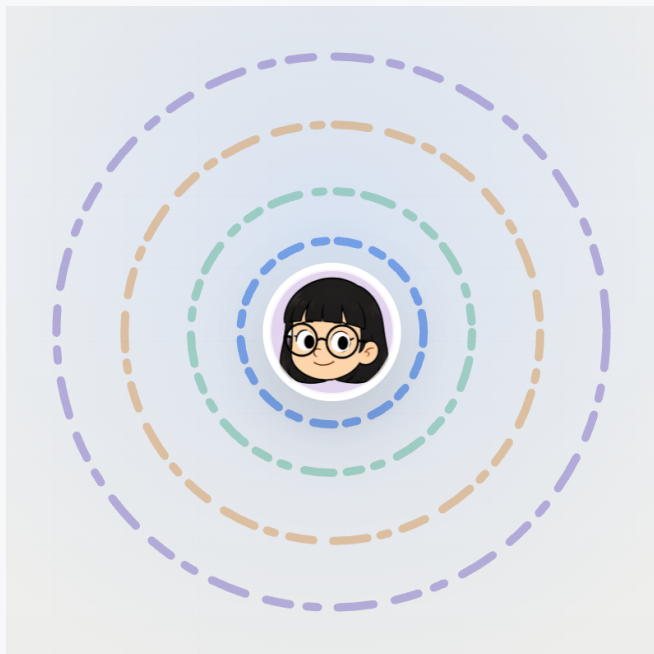
05

ヘッダー操作

設定とテーマ切替をヘッダー右上にまとめています。

画面カタログ

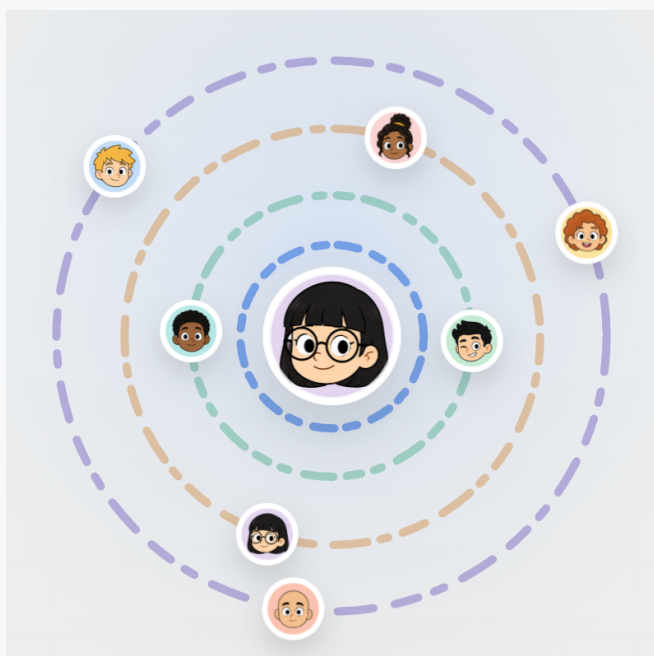
承認済み WebDrop 1.0.87 画面一覧



06

候補なしの軌道

近くの端末を表示する前の、4本の軌道パターンです。



07

候補を表示した軌道

候補を別々のリングに配置します。プロフィールは静止させ、軌道移動中も落ち着いて見えるようにします。

画面カタログ

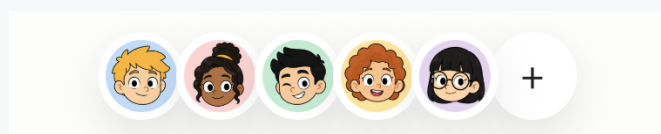
承認済み WebDrop 1.0.87 画面一覧



08

接続シート

相手を選ぶと下部シートが開き、右スワイプで接続を確定します。



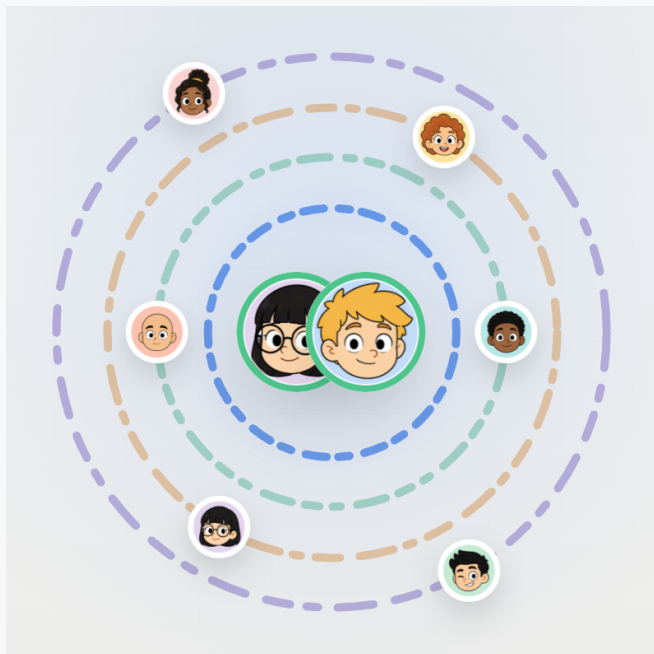
09

候補アイコン列

軌道と同じプロフィール表現を使い、円形の縁が見える小さな重なりで候補を並べます。

画面カタログ

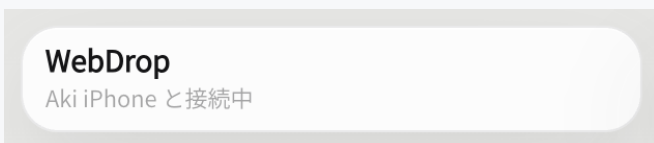
承認済み WebDrop 1.0.87 画面一覧



10

接続中のベン図表示

自分と相手を同じ大きさに重ねた静止プロフィールとして表示し、リングのアニメーションで接続状態を示します。



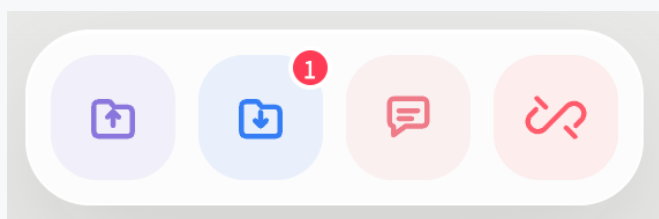
11

接続中ステータス

接続完了後は、接続している相手の名前をステータス行に表示します。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



12

接続後のドック

接続確認後にだけ、送信、受信、チャット、切断を表示します。



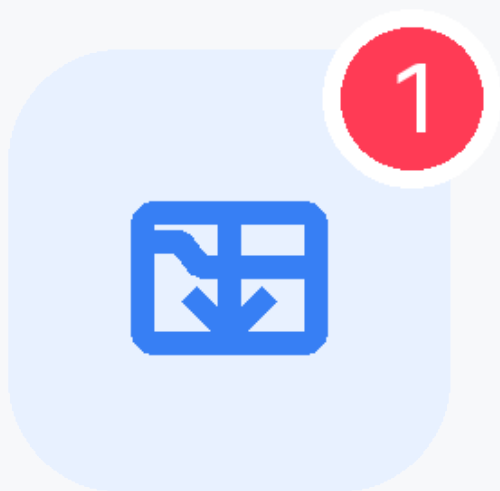
13

送信アイコン

ファイル送信シートを開きます。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



14

受信アイコン

受信ファイルを開き、利用可能なファイルがある場合はバッジを表示します。



15

チャットアイコン

接続相手とのチャットシートを開きます。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



16

切断アイコン

短い解除アニメーションの後で現在の接続を終了します。



17

ファイル選択前の送信シート

最初にファイルを選択します。ファイルが選ばれるまで送信スワイプは無効です。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



18

ファイル選択後の送信シート

選択したファイルを一覧に表示し、アイコンと文字が重ならない上スワイプ操作で送信します。



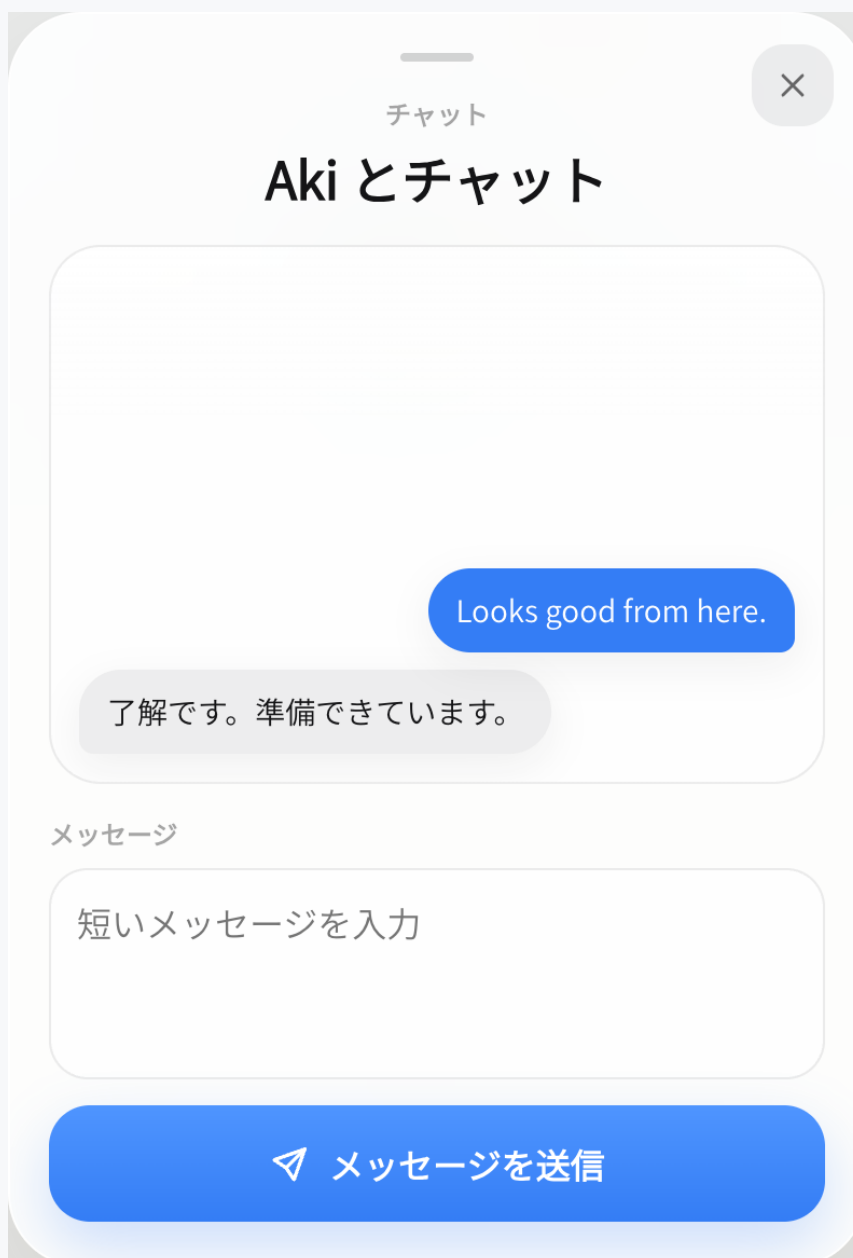
19

受信シート

受信したファイルを開く操作とともに表示し、ドックのバッジ件数を反映します。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧

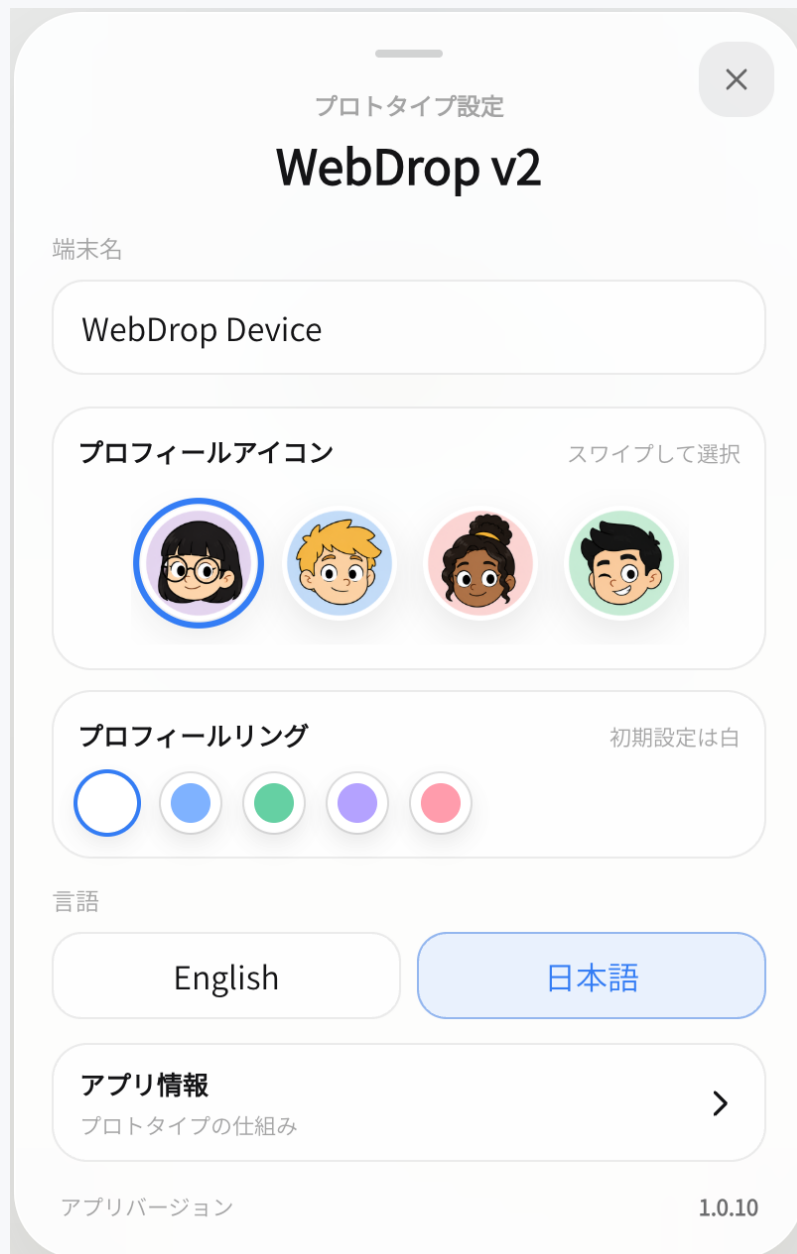


チャットシート

ファイル操作とは別の専用シートで短いメッセージを送ります。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



設定シート

プロフィール、リング、端末名、言語、アプリ情報をまとめています。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧

端末名

WebDrop Device

22

端末名入力

最後の1文字まで消しても勝手に初期値へ戻らず、自由に編集できます。

プロフィールアイコン

スワイプして選択



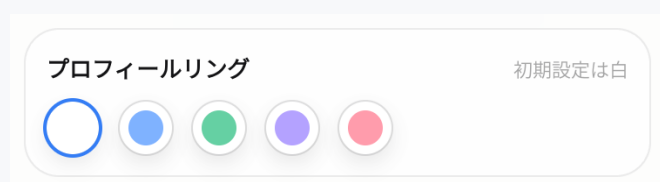
23

プロフィールアイコン選択

写真をアップロードする代わりに、用意されたキャラクターを横スワイプして選びます。

画面カタログ

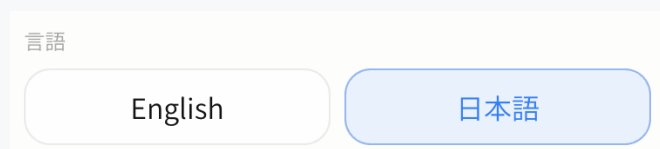
承認済み WebDrop 1.0.87 画面一覧



24

プロフィールリング選択

標準の白に加え、青、緑、紫、ローズのリング色を選択できます。



25

言語選択

アプリ内の全ラベルを英語と日本語で切り替えます。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧

アプリ情報

プロトタイプ仕組み



26

アプリ情報リンク

デザイン、技術構成、軌道アニメーションの詳細を別シートへ移動します。

アプリバージョン

1.0.10

27

アプリバージョン

設定画面の下部に現在のバージョン1.0.87を表示します。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧

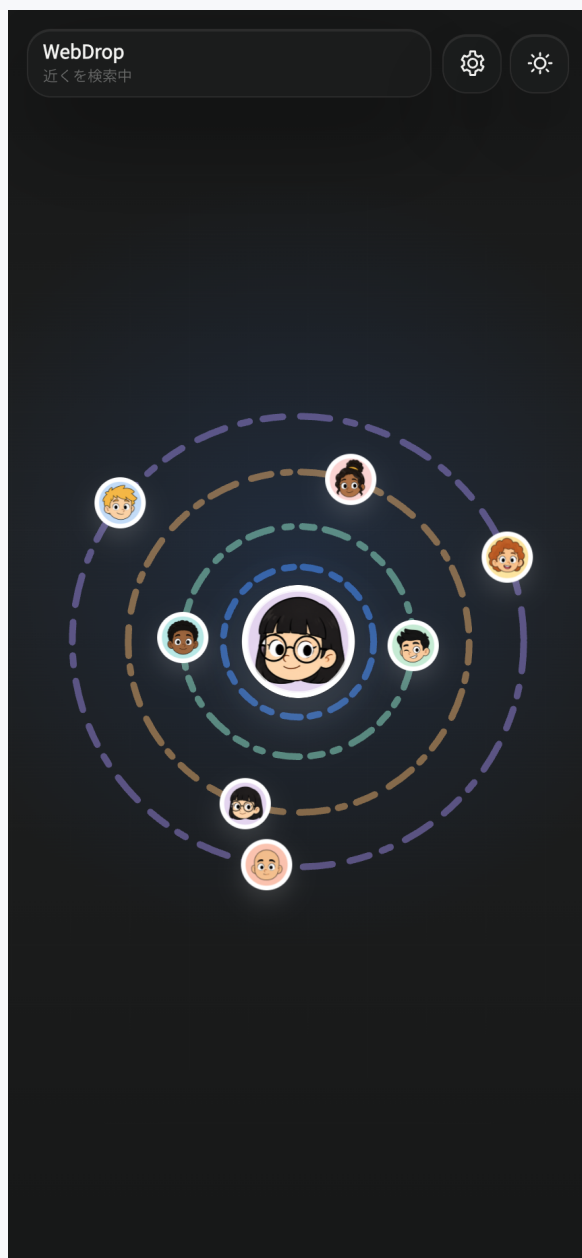


アプリ情報シート

設定画面を整理したまま、プロトタイプの説明と軌道アニメーション設定を表示します。

画面カタログ

承認済み WebDrop 1.0.87 画面一覧



ダークモードのホーム画面

同じ近隣端末UIを落ち着いたダークテーマで表示します。